

ORDERLY · SECURITY REVIEW

# Orderly

# Strategy Vault Updates.

A security review of Strategy Vault Updates.

| CLIENT  | PROJECT                | AUDITORS | AUDIT WINDOW                  | FINAL REPORT       |
|---------|------------------------|----------|-------------------------------|--------------------|
| Orderly | Strategy Vault Updates | Guardian | August 19 – September 5, 2025 | September 12, 2025 |

§ 00

# Contents.

56 pages — overview, confidence ranking, scope, 30 findings, disclaimer, and about Guardian.

|      |                                 |       |
|------|---------------------------------|-------|
| § 01 | Overview                        | P. 03 |
| § 02 | Confidence Ranking              | P. 04 |
| § 03 | Scope & Classifications         | P. 05 |
|      | Files in Scope                  | P. 06 |
| § 04 | Findings                        | P. 07 |
|      | Main Review · 21 findings       | P. 10 |
|      | Remediation Review · 9 findings | P. 41 |
| § 05 | Disclaimer                      | P. 55 |
| § 06 | About Guardian                  | P. 56 |

§ 01

# Overview.

Guardian reviewed Strategy Vault Updates across 2 review rounds. The final report includes 1 High, 5 Medium, 16 Low, 8 Informational findings.

|              |                               |                          |                                                            |
|--------------|-------------------------------|--------------------------|------------------------------------------------------------|
| AUDIT FIRM   | Guardian                      | REPOSITORY               | orderlynetwork/orderly-v2/strategy-vault                   |
| CLIENT       | Orderly                       | METHODOLOGY              | Smart contract · Manual · Main Review · Remediation Review |
| PROJECT      | Strategy Vault Updates        | MAIN REVIEW COMMIT       | c378880a5d7d                                               |
| PREPARED BY  | Guardian                      | REMEDATION REVIEW COMMIT | fdb4fd2e7284                                               |
| AUDIT WINDOW | August 19 – September 5, 2025 |                          |                                                            |
| FINAL REPORT | September 12, 2025            |                          |                                                            |

## 1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

|                     |                |                     |                 |                      |
|---------------------|----------------|---------------------|-----------------|----------------------|
| 1                   | 2              | 3                   | 4               | 5                    |
| VERY LOW CONFIDENCE | LOW CONFIDENCE | MODERATE CONFIDENCE | HIGH CONFIDENCE | VERY HIGH CONFIDENCE |

## 1.2 Findings at a Glance

| SEVERITY                     | TOTAL | PENDING | ACK | PARTIAL | RESOLVED |
|------------------------------|-------|---------|-----|---------|----------|
| <span>●</span> Critical      | 0     | 0       | 0   | 0       | 0        |
| <span>●</span> High          | 1     | 0       | 1   | 0       | 0        |
| <span>●</span> Medium        | 5     | 0       | 3   | 0       | 2        |
| <span>●</span> Low           | 16    | 0       | 14  | 0       | 2        |
| <span>●</span> Informational | 8     | 0       | 5   | 0       | 3        |
| All severities               | 30    | 0       | 23  | 0       | 7        |

## § 02

# Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

| CONFIDENCE RANKING                          | DEFINITION AND RECOMMENDATION                                                                                                                                                                                               | RISK PROFILE                                                                              |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>5 Very High Confidence</b>               | Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit. | 0 High/Critical findings and few Low/Medium severity findings.                            |
| <b>4 High Confidence</b><br><b>ASSIGNED</b> | Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.    | 0-1 High/Critical findings per engagement week and little to no Medium severity findings. |
| <b>3 Moderate Confidence</b>                | Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.                                                            | Some High/Critical findings, with various Medium severity findings.                       |
| <b>2 Low Confidence</b>                     | Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.                                                                     | Over 6 High and Critical findings across the engagement.                                  |
| <b>1 Very Low Confidence</b>                | The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.                                                                           | Over 12 High and Critical findings across the engagement.                                 |

§ 03

# Scope & Classifications.

Guardian reviewed Strategy Vault Updates in `orderlynetwork/orderly-v2/strategy-vault`.

## 3.1 Severity Matrix

|            | IMP · HIGH | IMP · MED | IMP · LOW |
|------------|------------|-----------|-----------|
| LIK · HIGH | Critical   | High      | Medium    |
| LIK · MED  | High       | Medium    | Low       |
| LIK · LOW  | Medium     | Low       | Low       |

IMPACT · HIGH  
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM  
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW  
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH  
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM  
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW  
Unlikely to ever occur in production.

# Files in Scope.

| FILE                             | NSLOC | TOTAL |
|----------------------------------|-------|-------|
| No files were recorded in scope. |       |       |

## § 04

# Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

| ID                 | TITLE                                                                        | CATEGORY            | SEVERITY | STATUS       | PAGE  |
|--------------------|------------------------------------------------------------------------------|---------------------|----------|--------------|-------|
| <b>MAIN REVIEW</b> |                                                                              |                     |          |              |       |
| M-01               | Claiming Can Bypass Required Cross-chain Fee                                 | Configuration       | MEDIUM   | RESOLVED     | p. 10 |
| M-02               | DEX Requests Are Marked "handled" Even When They Fail, Blocking Retries      | Logical Error       | MEDIUM   | ACKNOWLEDGED | p. 12 |
| M-03               | New Strategies Never Receive LP Deposits                                     | Configuration       | MEDIUM   | RESOLVED     | p. 14 |
| M-04               | allocateToFunds Partial DOS                                                  | Logical Error       | MEDIUM   | ACKNOWLEDGED | p. 16 |
| L-01               | Lack Of Deadline For Signatures                                              | Signatures          | LOW      | ACKNOWLEDGED | p. 17 |
| L-02               | Missing Source Chain Validation For Ledger-Originated Messages In _lzReceive | Validation          | LOW      | ACKNOWLEDGED | p. 18 |
| L-03               | Solana DEX ID Validation Ignores Payload Type                                | Validation          | LOW      | ACKNOWLEDGED | p. 20 |
| L-04               | Rounding In Multi-fund Allocation Causes Systematic Under/over Accounting    | Rounding            | LOW      | RESOLVED     | p. 22 |
| L-05               | Claim Fee Is Not Chain Specific                                              | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 24 |
| L-06               | handleDexRequests() Doesn't Verify Token                                     | Validation          | LOW      | ACKNOWLEDGED | p. 25 |
| L-07               | No Only-delegatecall Guard In LedgerExtension Contract                       | Validation          | LOW      | ACKNOWLEDGED | p. 26 |

## Findings continued.

| ID   | TITLE                                                                     | CATEGORY         | SEVERITY      | STATUS       | PAGE  |
|------|---------------------------------------------------------------------------|------------------|---------------|--------------|-------|
| L-08 | Integer Division Truncation in Cross-Chain Fee Calculation                | Rounding         | LOW           | ACKNOWLEDGED | p. 27 |
| L-09 | Duplicated Requests Dilute The ccFee                                      | Logical Error    | LOW           | ACKNOWLEDGED | p. 29 |
| L-10 | Incorrect feeRate Check                                                   | Validation       | LOW           | ACKNOWLEDGED | p. 31 |
| L-11 | Dex Requests Can Use Invalid Accounts                                     | Validation       | LOW           | ACKNOWLEDGED | p. 32 |
| I-01 | Ledger Layout Must Match                                                  | Upgradeability   | INFORMATIONAL | RESOLVED     | p. 33 |
| I-02 | Redundant uint128 Cast In _processDeposit                                 | Best Practices   | INFORMATIONAL | RESOLVED     | p. 34 |
| I-03 | Unnecessary Mapping Slot Derivations In _handleRequest                    | Gas Optimization | INFORMATIONAL | RESOLVED     | p. 35 |
| I-04 | depositNative Function Is Not payable                                     | Configuration    | INFORMATIONAL | ACKNOWLEDGED | p. 36 |
| I-05 | Double Withdraw Risk                                                      | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 38 |
| I-06 | ProtocolVaultLedger Relies On Off-chain Calculations And Operator Actions | Warning          | INFORMATIONAL | ACKNOWLEDGED | p. 39 |

### REMEDIATION REVIEW

|      |                                                    |                     |        |              |       |
|------|----------------------------------------------------|---------------------|--------|--------------|-------|
| H-01 | Rebalance Updates Are Undone By Next Settle        | Logical Error       | HIGH   | ACKNOWLEDGED | p. 41 |
| M-01 | SP price discrepancy between rebalance and deposit | Logical Error       | MEDIUM | ACKNOWLEDGED | p. 43 |
| L-01 | Decimal Conversion Applied To Withdraw Shares      | Warning             | LOW    | RESOLVED     | p. 45 |
| L-02 | Previous Rebalance Request Is Not Cleared          | Unexpected Behavior | LOW    | ACKNOWLEDGED | p. 47 |
| L-03 | Disabling strategy providers may DOS the Vault     | Configuration       | LOW    | ACKNOWLEDGED | p. 49 |

## Findings continued.

| ID   | TITLE                                                                 | CATEGORY            | SEVERITY      | STATUS       | PAGE  |
|------|-----------------------------------------------------------------------|---------------------|---------------|--------------|-------|
| L-04 | Rebalance can assign assets to a 0-ratio SP                           | Unexpected Behavior | LOW           | ACKNOWLEDGED | p. 50 |
| L-05 | Rebalanced SP can end up having shares, but no assets                 | Unexpected Behavior | LOW           | ACKNOWLEDGED | p. 51 |
| I-01 | quoteClaim Uses Param ChainId While Send Path Uses message.dstChainId | Validation          | INFORMATIONAL | ACKNOWLEDGED | p. 52 |
| I-02 | Unenforced Operation Order Enables Mis-sequenced Period Actions       | Configuration       | INFORMATIONAL | ACKNOWLEDGED | p. 54 |

MEDIUM **M-01**

RESOLVED

# Claiming Can Bypass Required Cross-chain Fee

CATEGORY Configuration

LOCATION ProtocolVault.sol

REVIEW Main Review

## DESCRIPTION

`ProtocolVault.updateUnClaimed` credits users with claimable assets and increments a per-user fee accumulator:

```
// ProtocolVault.updateUnClaimed(...)
userClaimedById[userId][USDC_HASH].unClaimedAssets += userClaimInfos[i].assets;
crossChainFee[userId] += ccFee;
```

However, the non-payable `claim()` path does not consult `crossChainFee` at all, it simply transfers the full amount and never clears any pending fee:

```
function claim(ClaimParams memory claimParams) external whenNotPaused {
// no check on crossChainFee[id]
uint256 amount = userClaimedById[id][USDC_HASH].unClaimedAssets;
userClaimedById[id][USDC_HASH].unClaimedAssets = 0;
delete userClaimedById[id][USDC_HASH].requestIds;
SafeTransferLib.safeTransfer(ERC20(claimParams.token), msg.sender, amount);
}
```

By contrast, `claimWithFee()` does enforce and clear the fee:

```
function claimWithFee(ClaimParams memory claimParams) external payable whenNotPaused {
require(msg.value == crossChainFee[id], "NotEnoughCCFee()");
...
delete crossChainFee[id];
}
```

This allows a user with `crossChainFee[id] > 0` to call `claim()` and receive their assets without paying the intended cross-chain fee. The fee then “dangles” in storage and can continue to accumulate across updates, creating an inconsistent state and a systematic loss of expected fee revenue. If a UI later uses `claimWithFee()`, exact-equality on the accumulated `crossChainFee[id]` may also cause confusing reverts when there is no remaining `unClaimedAssets`.

**M-01** Claiming Can Bypass Required Cross-chain Fee

CONTINUED

Therefore, users can repeatedly avoid paying cross-chain fees while still receiving their assets, causing protocol fee leakage and stale fee balances.

**RECOMMENDATION**

Enforce fee settlement whenever a fee is pending. The simplest hardening is to block the free path when a fee exists:

```
// In claim():  
require(crossChainFee[id] == 0, "Fee pending - use claimWithFee");
```

Alternatively, merge both flows into a single payable `claim()` that conditionally requires and clears the fee:

```
if (crossChainFee[id] > 0) { require(msg.value == crossChainFee[id], "fee mismatch"); delete  
crossChainFee[id]; }
```

Also consider clearing `crossChainFee[id]` only upon successful claims and documenting whether fees are per-claim or per-user to avoid future ambiguity.

**RESOLUTION**

Status: Resolved.

<https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/commit/ee4f6ef1dfdb68bf9e7203e87a76038d4ced2d4f>

MEDIUM **M-02**

ACKNOWLEDGED

# DEX Requests Are Marked “handled” Even When They Fail, Blocking Retries

CATEGORY Logical Error  
LOCATION ProtocolVaultLedger.sol  
REVIEW Main Review

## DESCRIPTION

In `handleDexRequests`, each item is flagged as handled before the contract knows whether the state change succeeded. The function sets `isDexRequestHandled[requestId] = true` and then calls `_handleRequest`. When `_handleRequest` returns `false` (typical for withdrawals if there aren't enough `pendingShares` or `pendingStrategyProviderShares` to freeze), the function only emits `DexWithdrawNotEnough(requestId)` but leaves the “handled” flag set. Because the request is now considered consumed, resubmitting the same `dexRequestId` later, after balances are updated by the period pipeline, will be rejected with an `AlreadyCalled` error and the withdrawal can never progress.

Minimal excerpt showing the ordering problem:

```
isDexRequestHandled[requestId] = true; // set too early
if (_handleRequest(request.dexRequestData.payloadType, request.id, tokenHash,
request.dexRequestData.amount) {
emit DexRequestHandled(request);
} else {
emit DexWithdrawNotEnough(requestId); // but requestId already consumed
}
```

This affects `LP_WITHDRAW` and `SP_WITHDRAW` flows, where `_handleRequest` checks `_checkWithdraw` against pending share balances and returns `false` when the operator has not yet staged enough pending shares for the id.

The main impact of this issue is that valid withdrawals can be permanently dropped due to timing, causing operational DoS and cross-chain accounting mismatches.

## RECOMMENDATION

Consider only marking a DEX request as handled after a successful state change, e.g.:

```
if (_handleRequest(...)) { isDexRequestHandled[requestId] = true; }
```

M-02

DEX Requests Are Marked "handled" Even When They Fail, Blocking Retries

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

# New Strategies Never Receive LP Deposits

CATEGORY Configuration

LOCATION LedgerCoreImpl.sol

REVIEW Main Review

## DESCRIPTION

In `LedgerCoreImpl.allocateToFunds`, the multi-strategy deposit path distributes `pendingLpDepositAssets` pro-rata to existing main vault exposure only, computed from each strategy's `mainShares` converted to assets. When a strategy is added later (e.g., at period N) it starts with `mainShares == 0`. The allocation loop first sums the current main exposure:

```
totalMainAssetsInFund += LedgerUtils._convertToAssets(  
    strategyFundToken.mainShares,  
    strategyFundToken.fundAssetsAfterFee,  
    strategyFundToken.totalShares,  
    Math.Rounding.Floor  
);
```

and then assigns each strategy's share:

```
uint256 mainAssetsInFund = LedgerUtils._convertToAssets(  
    strategyFundToken.mainShares,  
    strategyFundToken.fundAssetsAfterFee,  
    strategyFundToken.totalShares,  
    Math.Rounding.Floor  
);  
uint256 distributeDepositAssets =  
    depositAssets.mulDiv(mainAssetsInFund, totalMainAssetsInFund, Math.Rounding.Floor);
```

A newly added strategy therefore receives zero from LP deposits because `mainAssetsInFund == 0`. Self-funding via `SP_DEPOSIT` does not help: `_handleSPDeposit` mints strategy-provider shares (`pendingStrategyProviderShares`) and updates `pendingTotalShares/Assets`, but does not increase `pendingMainShares`, so the strategy's weight in future LP splits remains zero. Since `allocateToFunds` is allowed only once per period, operators cannot do a "seed then allocate" pattern in the same period.

**M-03** New Strategies Never Receive LP Deposits

CONTINUED

**RECOMMENDATION**

Incorporate a method for the operator to allocate a desired seed amount of main vault capital to a newly added strategy (e.g., "rebalance from existing strategies" to ensure the newcomer starts with a non-zero main share).

**RESOLUTION**

Status: Resolved.

[https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/merge\\_requests/35](https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/merge_requests/35)

MEDIUM **M-04**

ACKNOWLEDGED

# allocateToFunds Partial DOS

|          |                                                                                                                                                                                                                                                                                                                                                               |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Logical Error                                                                                                                                                                                                                                                                                                                                                 |
| LOCATION | <a href="https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/c378880a5d7ddcaf8c34c6a0a487982c9eddb057/contracts/Ledger/LedgerCoreImpl.sol#L244-L245">https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/c378880a5d7ddcaf8c34c6a0a487982c9eddb057/contracts/Ledger/LedgerCoreImpl.sol#L244-L245</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                                                                   |

## DESCRIPTION

`LedgerCoreImpl.allocateToFunds()` distributes all processed deposits and withdrawals to specified strategies. In case there is more than one strategy, funds will be split proportionally. The problem arises when there aren't any funds in the strategies, either because it's the start of the vault or because they have been naturally depleted in a previous period.

In that case, `totalMainAssetsInFund` will be computed as 0 and the following division will revert:

```
uint256 distributeDepositAssets = depositAssets.mulDiv(mainAssetsInFund,
totalMainAssetsInFund, Math.Rounding.Floor);
```

In result, allocating the funds to multiple strategies won't be possible and the Orderly operator will be forced to choose only one strategy for the given period.

## RECOMMENDATION

Split all the funds evenly between the strategies if `totalMainAssetsInFund == 0`

## RESOLUTION

Status: Acknowledged.

[https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/merge\\_requests/35](https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/merge_requests/35)

LOW **L-01**

ACKNOWLEDGED

# Lack Of Deadline For Signatures

CATEGORY Signatures  
LOCATION Global  
REVIEW Main Review

## DESCRIPTION

Signatures in the Ledger contract don't have deadline parameter which means they may be executed in any point in time. This can lead to unfavorable actions being executed, especially for dex requests.

## RECOMMENDATION

Consider adding deadline checks.

## RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

# Missing Source Chain Validation For Ledger-Originated Messages In `_lzReceive`

CATEGORY Validation  
LOCATION `VaultCrossChainManager.sol`  
REVIEW Main Review

## DESCRIPTION

`VaultCrossChainManager._lzReceive` accepts two ledger-originated message types, `ASSETS_DISTRIBUTION` and `UPDATE_USER_CLAIM` and immediately decodes and acts on them without asserting that they actually came from the ledger chain.

In the `ASSETS_DISTRIBUTION` branch the code decodes the payload and then calls into `ProtocolVault.depositToStrategy`. Similarly, in `UPDATE_USER_CLAIM` the code decodes and updates claims.

Neither branch validates that `strategyVaultCCmessage.srcChainId` corresponds to the configured ledger chain (nor cross-checks it against the LayerZero `Origin.srcEid`). While the inherited receiver already enforces that the message comes from a configured peer for a given EID, the contract may still be configured with multiple peers/EIDs. Without an explicit "ledger chain only" check here, any other configured peer could send these payload types and the handler would accept them, and the handler trusts the chain IDs embedded inside the message for decimal conversion. Impact: a misconfigured or compromised non-ledger peer could trigger unintended strategy deposits or mutate claim balances, and incorrect `src/dst` IDs inside the message could skew amount conversions.

## RECOMMENDATION

Defensively assert that these branches only execute for messages whose source is the ledger chain and, optionally, that the embedded `dstChainId` matches `block.chainid`. A minimal fix is to add a source-chain check that uses either a stored `ledgerChainId` or the EID mapping together with the LayerZero `Origin`:

```
require(chainIdToEid[strategyVaultCCmessage.srcChainId] == _origin.srcEid, "invalid ledger source");
```

Apply the same check in both `ASSETS_DISTRIBUTION` and `UPDATE_USER_CLAIM` branches. Optionally also assert `strategyVaultCCmessage.dstChainId == block.chainid` to prevent forged destination IDs affecting decimal conversion.

**L-02**      Missing Source Chain Validation For Ledger-Originated Messages In      **CONTINUED**  
             \_lzReceive

— **RESOLUTION**

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

# Solana DEX ID Validation Ignores Payload Type

CATEGORY Validation  
LOCATION LedgerExtension.sol  
REVIEW Main Review

## DESCRIPTION

In `LedgerExtension._verifySolRequest`, Solana DEX requests always validate the identifier (`request.id`) as an LP account ID, irrespective of the `payloadType`. The code path is:

```
function _verifySolRequest(DexRequest calldata request) internal view {
    DexRequestData calldata data = request.dexRequestData;
    // Verify id (always as Account ID)
    if (!VaultUtils.validateAccountId(data.receiver, vaultBroker[data.vaultId], request.id)) {
        revert InvalidId();
    }
    Signature.verifySOLSig(data, request.r, request.s, request.chainId, data.receiver);
}
```

For `SP_DEPOSIT` or `SP_WITHDRAW`, this means a legitimate strategy provider ID will be rejected because it is checked as an account ID. Conversely, a caller can submit an SP request using a valid account ID and pass `_verifySolRequest`. Execution then proceeds to `_handleRequest`; for example, the SP-withdraw branch:

```
} else if (payloadType == PayloadType.SP_WITHDRAW) {
    if (_checkWithdraw(amount, strategyFundToken.frozenShares,
        strategyFundToken.pendingState.pendingStrategyProviderShares)) {
        strategyFundToken.frozenShares += amount;
    } else {
        return false;
    }
}
```

Because the identifier that passed validation is an account ID (not a strategy provider ID), the SP share bucket is zero and `_checkWithdraw` fails; however, in `handleDexRequests` the request is marked as handled before invoking `_handleRequest`:

## L-03 Solana DEX ID Validation Ignores Payload Type

CONTINUED

```
isDexRequestHandled[requestId] = true; // set before attempting the state change
if (_handleRequest(...)) { ... } else { ... } // failure does not unset the flag
```

This combination produces two failures: first, legitimate SP operations on Solana are blocked by incorrect on-chain ID validation; second, a mismatched SP request that slips through the validator burns the `dexRequestId` (since it is flagged as handled even when `_handleRequest` returns `false`), preventing a corrected re-submission with the same ID and forcing off-chain remediation.

### — RECOMMENDATION

Validate the identifier according to `payloadType` and only mark `isDexRequestHandled` after a successful state transition. Concretely, in `_verifySolRequest` branch on the payload:

```
if (data.payloadType == PayloadType.LP_DEPOSIT || data.payloadType == PayloadType.LP_WITHDRAW)
{
  require(VaultUtils.validateAccountId(data.receiver, vaultBroker[data.vaultId], request.id),
    "InvalidId");
} else if (data.payloadType == PayloadType.SP_DEPOSIT || data.payloadType ==
PayloadType.SP_WITHDRAW) {
  require(VaultUtils.validateSPId(protocolVault /* or SP namespace */,
    vaultBroker[data.vaultId], request.id), "InvalidId");
} else {
  revert InvalidType();
}
```

Then, in `handleDexRequests`, assign `isDexRequestHandled[requestId] = true` only if `_handleRequest` returns `true`; if it returns `false`, revert (or store a separate “failed” status) so the DEX can correct and resubmit without losing the request ID.

### — RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

# Rounding In Multi-fund Allocation Causes Systematic Under/over Accounting

CATEGORY Rounding  
LOCATION LedgerCoreImpl.sol  
REVIEW Main Review

## DESCRIPTION

In the multi-fund branch of `LedgerCoreImpl.allocateToFunds`, the contract splits deposits and withdrawals proportionally across strategies using different rounding directions per fund and does not perform any balancing step afterward. For deposits, the per-fund split uses `Math.Rounding.Floor`:

```
uint256 distributeDepositAssets =  
depositAssets.mulDiv(mainAssetsInFund, totalMainAssetsInFund, Math.Rounding.Floor);
```

Every slice is rounded down, so the sum of all `distributeDepositAssets` is  $\leq$  `pendingLpDepositAssets`, leaving unassigned "dust" that never reaches any strategy's `pendingTotalAssets` in that period. For withdrawals, the split uses `Math.Rounding.Ceil`:

```
uint256 distributeWithdrawAssets =  
withdrawAssets.mulDiv(mainAssetsInFund, totalMainAssetsInFund, Math.Rounding.Ceil);
```

Every slice is rounded up, so the sum of all `distributeWithdrawAssets` is  $\geq$  `pendingLpWithdrawAssets` and the contract subtracts more from strategies' `pendingTotalAssets` than the requested withdrawal. Over time, repeated floors on deposits and ceils on withdrawals create a systematic downward drift in per-fund asset totals, which can bias NAV/share pricing. As a simple illustration with three equal funds and 100 units: floor on deposits yields  $33+33+33=99$  (1 unit not allocated), while ceil on withdrawals yields  $34+34+34=102$  (2 units over-removed). In edge cases with many funds and very small amounts, the per-fund over-subtractions can even trigger underflow reverts.

## RECOMMENDATION

Use an exact-sum allocation scheme so that per-fund amounts add up exactly to the total for both deposits and withdrawals. Two robust approaches are:

- Largest-remainder method: compute exact rationals, take floors for all, then assign the remaining "dust" to funds with largest fractional remainders.

## L-04 Rounding In Multi-fund Allocation Causes Systematic Under/over Accounting

CONTINUED

• Balance-on-last: floor the first N-1 allocations and set the last allocation to `total - sumFloors`. For example, for the last fund: `distributeAssets_last = totalAssets - sumAllocatedSoFar;`

### — RESOLUTION

Status: Resolved.

[https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/merge\\_requests/35](https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/merge_requests/35)

LOW **L-05**

ACKNOWLEDGED

# Claim Fee Is Not Chain Specific

CATEGORY Unexpected Behavior  
LOCATION LedgerCoreImpl.sol#L537  
REVIEW Main Review

## DESCRIPTION

`LedgerCoreImpl.updateUnclaimed()` quotes LayerZero for the native fee that must be paid on the Orderly chain. It will then encode the amount in the crosschain message sent to the Vault side where it will be recorded in `crossChainFee` and will be paid by the user upon claim. However, if the vault chain has a different native token than the Orderly chain, this action will result in accounting mismatch. For example, if the contracts are deployed on Polygon PoS, the fee recorded will be in MATIC instead of ETH.

## RECOMMENDATION

Add additional logic that handles these cases if you decide to deploy to such chains.

## RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

# handleDexRequests ( ) Doesn't Verify Token

CATEGORY Validation  
LOCATION LedgerExtension.sol  
REVIEW Main Review

## DESCRIPTION

`LedgerExtension.handleDexRequests()` doesn't check if the token provided is USDC, which makes it possible to use the system with other tokens even if they are not supported.

## RECOMMENDATION

Skip any requests if the token they are working with is not `USDC`.

## RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

# No Only-delegatecall Guard In LedgerExtension Contract

|          |                     |
|----------|---------------------|
| CATEGORY | Validation          |
| LOCATION | LedgerExtension.sol |
| REVIEW   | Main Review         |

## — DESCRIPTION

`LedgerExtension` is designed to be used only via `delegatecall` from `ProtocolVaultLedger`, but it exposes public/external entry points without enforcing that call context. If someone calls the implementation address of `LedgerExtension` directly, its functions execute against the implementation's own storage, not the ledger's storage, and can still emit "successful" events (e.g., `OperationHandled`, `DexRequestHandled`) and update mappings like `isDexRequestHandled` within the implementation contract. This does not mutate the real ledger state but can pollute on-chain telemetry and generally widens the surface for future mistakes. The risk is currently limited to observability and monitoring confusion because state changes remain isolated to the implementation contract.

## — RECOMMENDATION

Add an only-delegatecall guard to every external entry point in `LedgerExtension` (and mirror it in other implementation modules) so that calls revert unless executed via the proxy. A minimal pattern is to record the implementation's own address once and require that `address(this)` differs when invoked (i.e., reject direct calls): `require(address(this) != IMPLEMENTATION_SELF, "impl: only delegatecall");`.

## — RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

# Integer Division Truncation in Cross-Chain Fee Calculation

|          |                    |
|----------|--------------------|
| CATEGORY | Rounding           |
| LOCATION | LedgerCoreImpl.sol |
| REVIEW   | Main Review        |

## DESCRIPTION

In the `LedgerCoreImpl` contract's `updateUnclaimed` function, the cross-chain fee `ccFee` obtained from the `IVaultCrossChainManager`'s `quoteClaim` function is divided by the number of valid unclaimed requests `len` before encoding it into the `StrategyVaultCCMessage` payload as `abi.encode(periodId, ccFee / len, userClaimInfos)`. Solidity performs integer division on `uint256` types by default flooring the result and discarding any remainder, meaning that for non-divisible values the computed per-claim fee will be lower than intended. For instance, if `ccFee` equals 10 and `len` equals 3, then `ccFee / len` evaluates to 3 with a remainder of 1 lost, resulting in the payload implying a total fee of 3 multiplied by 3 equaling 9 which is less than the originally quoted 10. This truncation happens because the division operator in Solidity does not round up or preserve fractions, leading to a systematic underestimation when the fee is not perfectly divisible by the batch size. The code snippet illustrating this is:

```
(ccFee,) = IVaultCrossChainManager(crossChainManager).quoteClaim(chainId, message);
message = _createCCMessage(
    PayloadType.UPDATE_USER_CLAIM, chainId, abi.encode(periodId, ccFee / len, userClaimInfos)
);
```

Here, the quoted `ccFee` is for the entire batch message, but the encoded per-claim value may cause the destination chain's processing logic to reconstruct a total that is insufficient if it multiplies back by `len`. The impact is potential underfunding of cross-chain messages leading to delivery failures, reverts.

## RECOMMENDATION

Replace the division with a ceiling calculation using `(ccFee + len - 1) / len` to ensure the total implied fee when multiplied back meets or exceeds the quoted amount, and add a check to revert if the per-claim fee is zero for non-empty batches.

---

**L-08** Integer Division Truncation in Cross-Chain Fee CalculationCONTINUED

---

**— RESOLUTION**

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

# Duplicated Requests Dilute The ccFee

|          |                                                                                                                                                                                                                                                                                                                                                     |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Logical Error                                                                                                                                                                                                                                                                                                                                       |
| LOCATION | <a href="https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/c378880a5d7ddcaf8c34c6a0a487982c9eddb057/contracts/Ledger/LedgerCoreImpl.sol#L471">https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/c378880a5d7ddcaf8c34c6a0a487982c9eddb057/contracts/Ledger/LedgerCoreImpl.sol#L471</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                                                         |

## — DESCRIPTION

`LedgerCoreImpl.updateUnclaimed()` is designed to handle duplicates in the `requestIds` passed by the BE. This becomes evident from the second check for a valid request.

```
// Ignore if handled
if (!_isValidRequestId(requestIds[i])) {
  userClaimInfos[index] = userClaimInfo[requestIds[i]];
  isUserClaimHandled[requestIds[i]] = true;
  index++;
  delete userClaimInfo[requestIds[i]];
}
```

Even though these requests won't be included in the `userClaimInfos` array, its length has already been set to `len` which includes all the duplicates.

In result, `userClaimInfos` will contain empty slots instead of duplicates, but there is a bigger problem - the fee calculation: `ccFee / len`

Since `len` includes the empty slots, the individual fee paid for each request will be diluted. Nobody will pay the fees for the empty slots which means they have to be covered by the Orderly protocol.

In addition, on the EVM side the `userClaimedById` and `crossChainFee` mappings will be populated for `address(0)` because of these empty slots.

## — RECOMMENDATION

The simplest solution is to compute the individual fee to be paid as `ccFee / index` instead of `ccFee / len`. While this won't stop the mappings being incorrectly populated, the fee dilution problem will be solved.

If you want to fix both issues, you should redesign the function.

---

**L-09** Duplicated Requests Dilute The ccFee

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

# Incorrect feeRate Check

|          |                              |
|----------|------------------------------|
| CATEGORY | Validation                   |
| LOCATION | ProtocolVaultLedger.sol: 277 |
| REVIEW   | Main Review                  |

## DESCRIPTION

Additional check that ensures fee cannot be changed before NAV is updated was added to the `setFeeRate()` function in response to the following finding from a previous audit round - <https://www.notion.so/guardianaudits/M-06-c8c88308a0b845d596cdca21c15f437a>.

However, in the latest commit, the check was modified to work for the previous period, not the current one

```
if (latestPeriodId != 0 && !isUpdateStrategyFundAssets[latestPeriodId - 1]) {  
    revert NotAllowedTime();  
}
```

For the vault to work correctly, once `updatePeriodId()` is called, it means `updateStrategyFundAssets()` has already been called for the previous period. This means the check in `setFeeRate()` will be always passing, reintroducing the issue from the original report. An exception is if `setFeeRate()` is called in period 1 because `updateStrategyFundAssets()` may have not been called in period 0. In that case, the fee can't be changed until period 1 ends.

## RECOMMENDATION

Check for the current period, not the previous one.

```
- if (latestPeriodId != 0 && !isUpdateStrategyFundAssets[latestPeriodId - 1]) {  
+ if (latestPeriodId != 0 && !isUpdateStrategyFundAssets[latestPeriodId]) {  
    revert NotAllowedTime();  
}
```

## RESOLUTION

Status: Acknowledged.

<https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/commit/61383b13c63840dce6d466a0a5cf5bb06011834e>

LOW **L-11**

ACKNOWLEDGED

# Dex Requests Can Use Invalid Accounts

CATEGORY Validation  
LOCATION LedgerExtension.sol#L132-155  
REVIEW Main Review

## DESCRIPTION

`LedgerExtension.handleDexRequests()` performs validation to confirm the account id to be modified is derived by the user address and the `brokerHash`. The `brokerHash` is fetched from `vaultBroker[data.vaultId]` using the `data.vaultId` field. However, there is no further validation applied to `brokerHash`. For example, it may be empty. This will yield invalid account id which is still controlled by the user and can be claimed by them, but is not derived from a valid broker. In result, users are able to bypass the accounts system and operate with invalid accounts.

It's also possible that `vaultBroker` for that `vaultId` was changed to `address(0)` by calling `setVaultBroker()`. One would expect accounts with that broker won't be used anymore, but the issue shows the opposite.

## RECOMMENDATION

Ensure `brokerHash` is not empty after fetching it in `_verifyEVMRequest` and `_verifySolRequest()`

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

RESOLVED

# Ledger Layout Must Match

|          |                |
|----------|----------------|
| CATEGORY | Upgradeability |
| LOCATION | Global         |
| REVIEW   | Main Review    |

## DESCRIPTION

All of the three contracts - `ProtocolVaultLedger`, `LedgerCoreImpl` and `LedgerExtension` - must have the same storage layout because the first one delegate calls the other two. The current layout is fine because none of the other contracts `ProtocolVaultLedger` inherits from has any state.

## RECOMMENDATION

Be careful with future updates since they can corrupt the Ledger storage layout.

## RESOLUTION

Status: Resolved.

<https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/commit/61383b13c63840dce6d466a0a5cf5bb06011834e>

## INFORMATIONAL I-02

RESOLVED

# Redundant uint128 Cast In \_processDeposit

CATEGORY Best Practices  
LOCATION VaultAdapter.sol  
REVIEW Main Review

## DESCRIPTION

In `VaultAdapter._processDeposit`, the `tokenAmount` field of `VaultDepositFE` is assigned using an explicit cast:

```
VaultDepositFE memory depositData = VaultDepositFE({  
  accountId: id,  
  brokerHash: brokerHash,  
  tokenHash: adapterDeposit.tokenHash,  
  tokenAmount: uint128(adapterDeposit.amount) // amount is already uint128  
});
```

If `adapterDeposit.amount` is indeed `uint128`, this cast is redundant and slightly reduces readability while adding a negligible cost.

## RECOMMENDATION

Remove the cast as `adapterDeposit.amount` is already `uint128`.

## RESOLUTION

Status: Resolved.

<https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/commit/ee4f6ef1dfdb68bf9e7203e87a76038d4ced2d4f>

## INFORMATIONAL I-03

RESOLVED

# Unnecessary Mapping Slot Derivations In `_handleRequest`

CATEGORY Gas Optimization  
LOCATION LedgerExtension.sol  
REVIEW Main Review

## DESCRIPTION

In `_handleRequest`, the function pre-derives both storage pointers:

```
AccountToken storage accountToken = accountTokenInfo[id][tokenHash];  
StrategyFundToken storage strategyFundToken = strategyFundTokenInfo[id][tokenHash];
```

irrespective of `payloadType`. In the `LP_*` branches only `accountToken` is used; in the `SP_*` branches only `strategyFundToken` is used. While this does not perform an SLOAD by itself, deriving a nested-mapping storage pointer requires one or more `keccak256` computations to obtain the slot. Computing the unused pointer on every call therefore incurs avoidable gas for each request (e.g., ~two extra `keccak256` operations per call in typical paths), with no functional benefit.

## RECOMMENDATION

Compute the storage pointer lazily inside the relevant branch only. For example, declare `AccountToken storage a = accountTokenInfo[id][tokenHash];` only in `LP_*` branches and `StrategyFundToken storage s = strategyFundTokenInfo[id][tokenHash];` only in `SP_*` branches. This avoids deriving unused mapping slots and saves gas on every request.

## RESOLUTION

Status: Resolved.

<https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/commit/61383b13c63840dce6d466a0a5cf5bb06011834e>

INFORMATIONAL **I-04**

ACKNOWLEDGED

# depositNative Function Is Not payable

|          |                  |
|----------|------------------|
| CATEGORY | Configuration    |
| LOCATION | VaultAdapter.sol |
| REVIEW   | Main Review      |

## DESCRIPTION

`VaultAdapter.depositNative` forwards native currency to the DEX vault:

```
function depositNative(AdapterDeposit memory adapterDeposit, bytes calldata signature)
external onlyOperator {
    (uint256 fee, VaultDepositFE memory depositData) = _processDeposit(adapterDeposit,
signature);
    uint256 totalValue = adapterDeposit.amount + fee;
    IDexVault(dexVault).depositTo{value: totalValue}(adapterDeposit.receiver, depositData);
    emit DepositFromCeffu(adapterDeposit, true);
}
```

However, the function is not marked `payable`. As a result, the caller cannot attach ETH to this transaction to fund `totalValue`. The call to `depositTo{value: totalValue}` will revert if the adapter contract's balance is insufficient, effectively causing a denial of service for native deposits unless the contract is pre-funded out-of-band via `receive()`. This contradicts the function's intent ("Deposit native token (ETH) to the specified receiver") and makes operational flows brittle, since success depends on prior top-ups and exact balance management.

Moreover, this forces the top-ups and deposits to be executed atomically otherwise a different operator could make use through another deposit call of the native assets that were sent but not yet consumed by the contracts.

## RECOMMENDATION

Consider declaring the function `payable` and validate the provided value against the required total:

```
function depositNative(AdapterDeposit memory adapterDeposit, bytes calldata signature)
external payable onlyOperator {
    (uint256 fee, VaultDepositFE memory depositData) = _processDeposit(adapterDeposit,
signature);
```

**I-04** depositNative Function Is Not payable

CONTINUED

```
uint256 totalValue = adapterDeposit.amount + fee;  
require(msg.value == totalValue, "Incorrect msg.value");  
IDexVault(dexVault).depositTo{value: msg.value}(adapterDeposit.receiver, depositData);  
}
```

If you prefer to allow overpayment, require `msg.value >= totalValue` and refund the excess before calling `depositTo`.

**— RESOLUTION**

Status: Acknowledged.

INFORMATIONAL **I-05**

ACKNOWLEDGED

# Double Withdraw Risk

|          |                    |
|----------|--------------------|
| CATEGORY | Informational      |
| LOCATION | LedgerCoreImpl.sol |
| REVIEW   | Main Review        |

## DESCRIPTION

After internal DEX withdraw happens, `handleDexRequests()` will increase the user's `frozenShares`, then `updateLPAndStrategyFund` will finalize the withdrawal, but it will also populate `userClaimInfo` for that `requestId`.

The BE should differentiate between normal requests and DEX requests and not call `updateUnclaimed()` for the DEX requests, but it's still technically possible. If this happens users will receive their funds twice - once by internal transfer and once by claiming from the ProtocolVault on the EVM chain.

## RECOMMENDATION

To avoid the risk completely add a check to `updateLPAndStrategyFund()` that skips writing to `userClaimInfo` if the current request is a DEX one.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-06

ACKNOWLEDGED

# ProtocolVaultLedger Relies On Off-chain Calculations And Operator Actions

CATEGORY Warning  
LOCATION ProtocolVaultLedger.sol  
REVIEW Main Review

## DESCRIPTION

The ledger's critical state transitions depend on values computed and orchestrated off-chain by privileged actors (the engine, operator, DEX signers and the cross-chain manager). On-chain code primarily verifies signatures and roles, not the economic correctness of the supplied data. For example, in `LedgerCoreImpl.updateStrategyFundAssets` the engine signs the per-strategy `totalAssets` (NAV) array that drives performance-fee minting and share-price updates; the contract only calls `Signature.verifyUpdateFundAssets(...)` and then computes fees and `fundAssetsAfterFee` from those off-chain NAVs. If the NAVs are stale or manipulated, performance fees and main/share accounting are affected with no on-chain way to self-correct. In `LedgerCoreImpl.updateLPAndStrategyFund` the engine submits the list of operations (LP/SP deposits/withdrawals, amounts and requestIds). The contract verifies the signature and applies conversions and freezes, assuming the off-chain stream correctly reflects real deposits/withdrawals observed on other chains; misuse or bugs can mis-credit `unAllocatedAssets` or freeze the wrong amounts until later settlement. In `LedgerCoreImpl.allocateToFunds` the operator chooses which strategy IDs participate in the split for the period; the on-chain logic does proportional math but does not enforce any fairness or inclusion invariants, so a strategy can be starved or over-favored by off-chain selection. In `LedgerCoreImpl.distributeAssets` the engine dictates the cross-chain distribution tuple `(chainId, assets)` per strategy and the ledger blindly constructs and sends cross-chain messages through the configured manager; there is no on-chain reconciliation that the distribution matches pending balances. Finally, in `LedgerCoreImpl.updateUnclaimed` the engine provides the set of `requestIds` to finalize and the contract clears and forwards them; omission or mis-ordering off-chain leaves claims pending indefinitely. Similar dependence exists in `LedgerExtension.handleDexRequests`, where the contract trusts DEX-provided signatures and request formatting to mutate `unAllocatedAssets` or `frozenShares`. These flows are role- and signature-gated, but economic truth (correct NAVs, operation completeness, fair inclusion, and accurate distributions) is externalized to off-chain processes.

---

**I-06** ProtocolVaultLedger Relies On Off-chain Calculations And Operator Actions **CONTINUED**

---

**— RECOMMENDATION**

Merely informative. Note that any error in the offchain components of the protocol can have a severe impact in the protocol.

**— RESOLUTION**

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

# Rebalance Updates Are Undone By Next Settle

CATEGORY Logical Error  
LOCATION ProtocolVaultLedger.sol  
REVIEW Remediation Review

## DESCRIPTION

`LedgerExtension.rebalance` directly edits the final per-fund fields ( `totalAssets` , `totalShares` , `mainShares` ) but does not update `pendingState.*` or `fundAssetsAfterFee` . Later, the period pipeline calls `settleMainAndStrategyFunds` , which commits final fields from the pending state, thereby clobbering the earlier rebalance edits. During the window between `rebalance` and `settleMainAndStrategyFunds` , other flows (notably `allocateToFunds` ) read a mixed source of truth. Updated `mainShares / totalShares` paired with stale `fundAssetsAfterFee` , which can misweight allocations and make conversions inconsistent with the intended post-rebalance basis.

Settle overwrites final fields from pending:

```
// LedgerCoreImpl.settleMainAndStrategyFunds(...)
strategyFundToken.totalShares = pendingState.pendingTotalShares;
strategyFundToken.totalAssets = pendingState.pendingTotalAssets;
strategyFundToken.mainShares = pendingState.pendingMainShares;
```

Allocation uses an inconsistent triplet because `rebalance` doesn't touch `fundAssetsAfterFee` or `pendingState` :

```
// LedgerCoreImpl.allocateToFunds(...)
mainAssetsInFund = LedgerUtils._convertToAssets(
strategyFundToken.mainShares, // updated by rebalance()
strategyFundToken.fundAssetsAfterFee, // NOT updated by rebalance()
strategyFundToken.totalShares, // updated by rebalance()
...
);
```

Because of this, the intended cross-fund redistribution performed by `rebalance` does not persist across settlement and before settlement it can skew per-fund weights and conversions.

---

**H-01** Rebalance Updates Are Undone By Next SettleCONTINUED

---

**— RECOMMENDATION**

Consider applying rebalance deltas to `pendingState.pendingTotalAssets`, `pendingState.pendingTotalShares` and `pendingState.pendingMainShares` (and, if your pricing depends on it, reconcile `fundAssetsAfterFee`).

**PROOF OF CONCEPT** <https://gist.github.com/GuardianAudits/aa4147358b6aff4e042ab16436af32a0>

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

# SP price discrepancy between rebalance and deposit

CATEGORY Logical Error  
LOCATION LedgerExtension.sol#L178 LedgerCoreImpl.sol#L613  
REVIEW Remediation Review

## DESCRIPTION

During a `rebalance()`, the `fund.lastSharePrice` variable is used to calculate the main shares to be added or removed.

```
uint256 sharePrice = fund.lastSharePrice;  
...  
deltaShares = deltaAssets * 10 ** USDC_DECIMAL / sharePrice;
```

This price is modified at the beginning of the period when uploading the NAV via

```
updateStrategyFundAssets()  
  
strategyFundToken.lastSharePrice = (fundAssets - performanceFee) * 10 ** USDC_DECIMAL /  
fundShares;
```

However, `_handleSPDeposit` converts the deposited assets to shares using the current price of the SP fund, not the cached one.

```
uint256 depositShares = LedgerUtils._convertToShares(  
amount, strategyFundToken.fundAssetsAfterFee, strategyFundToken.totalShares,  
Math.Rounding.Floor  
);
```

In case `fundAssetsAfterFee` and `totalShares` are 0, shares and assets are priced 1:1. This discrepancy between the SP share price used in `rebalance` and the SP share price in `_handleSPDeposit` will cause accounting mismatch and lead to unfair funds distribution. For example:

1. LP deposits 100 USDC and receives 100 shares
2. SP deposits 100 USDC to `SP_B` and receives 100 shares
3. The LP funds are allocated to `SP_B` (mainShares = 100, totalShares = 200, assets = 200)
4. At the beginning of the next period the NAV is updated and `SP_B` reports a double increase, i.e. `assets` = 400 and `lastSharePrice` = 2

**M-01** SP price discrepancy between rebalance and deposit

CONTINUED

5. The SP withdraws all their shares from `SP_B` and receives 200 USDC (mainShares = 100, totalShares = 100, assets = 200)
6. Rebalance moves all of the LP's funds to `SP_A`. This leaves `SP_B` empty
7. In the next period the SP deposits again 100 USDC to `SP_B`. Since it's empty, they receive 100 shares.
8. Rebalance moves all of the LP's funds to `SP_B`. It uses `lastSharePrice = 2` and converts 200 USDC to 100 shares.
9. Now the LP and SP have both 100 shares, or 150 USDC to withdraw.
10. In the end, the SP stole 50 USDC from the LP.

**— RECOMMENDATION**

Consider if caching the `lastSharePrice` is needed instead of using the same ratio as `spDeposit` and `spWithdraw`

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/spPriceRebalancePOC/test/RebalanceSPPriceTest.t.sol>

**— RESOLUTION**

Status: Acknowledged.

<https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/commit/56cc904c9c6f13f3daa7dfe7e2917ea592d1aecf>

LOW **L-01**

RESOLVED

# Decimal Conversion Applied To Withdraw Shares

CATEGORY Warning  
LOCATION VaultCrossChainManager.sol  
REVIEW Remediation Review

## DESCRIPTION

In `_lzReceive` of `VaultCrossChainManager`, the incoming `OperationData.amount` is always passed through decimal conversion whenever `isSpecialDecimal[tokenHash][srcChainId]` is set, regardless of the operation type. This is correct for deposits (where `amount` is in asset units) but wrong for withdrawals, where `amount` represents shares, a dimensionless value that must not be scaled by token decimals.

Relevant code (current behavior):

```
// _lzReceive(...)
OperationData memory operationData = abi.decode(payload, (OperationData));
uint256 srcChainId = strategyVaultCCmessage.srcChainId;
if (isSpecialDecimal[operationData.tokenHash][srcChainId]) {
    operationData.amount = _convertAmount(
        operationData.amount,
        tokenDecimals[operationData.tokenHash][srcChainId],
        ledgerDecimal
    );
}
```

```
IProtocolVaultLedger(ledger).handleOpFromVault(payloadType, srcChainId, operationData);
```

For `LP_WITHDRAW` / `SP_WITHDRAW`, this scaled value is then treated as shares by the ledger:

- Intake freezes that (now wrong) amount of shares:

```
// LedgerExtension._handleRequest
if (payloadType == PayloadType.LP_WITHDRAW) {
    if (_checkWithdraw(amount, accountToken.frozenShares, accountToken.pendingShares)) {
        accountToken.frozenShares += amount; // amount is assumed to be shares
    } else {
        return false;
    }
}
```

**L-01** Decimal Conversion Applied To Withdraw Shares

CONTINUED

```
}  
}
```

- Settlement later requires the amount to be `<= frozenShares` and uses it to compute assets:

```
// LedgerCoreImpl._handleLpWithdraw  
LedgerUtils.requireEnoughFrozenShares(amount, accountToken.frozenShares);  
uint256 withdrawAssets = LedgerUtils._convertToAssets(amount, mainAssetsAfterFee, mainShares,  
Math.Rounding.Floor);
```

Because `_lzReceive` has down- or up-scaled shares, the ledger freezes the wrong number of shares. In practice, operators will typically settle an amount that matches what the ledger froze to avoid reverts, which produces silent under-withdrawals (e.g., user intended 600 shares; only 6 were frozen and settled). If the operator instead tries to settle the original amount, settlement reverts with `NotEnoughFrozenShare`, creating a DoS for that request. In short, the unit mismatch corrupts withdraw flows and can either underpay users or block settlement.

**— RECOMMENDATION**

Only perform decimal conversion for deposits (asset units). Do not convert `operationData.amount` for `LP_WITHDRAW` / `SP_WITHDRAW`, since it represents shares. A minimal fix is to gate the conversion:

```
// Pseudocode inside _lzReceive before calling handleOpFromVault  
if ((payloadType == LP_DEPOSIT || payloadType == SP_DEPOSIT) &&  
isSpecialDecimal[operationData.tokenHash][srcChainId]) {  
operationData.amount = _convertAmount(operationData.amount,  
tokenDecimals[operationData.tokenHash][srcChainId],  
ledgerDecimal);  
}  
// For withdraws, forward amount unchanged.
```

**— RESOLUTION**

Status: Resolved.

<https://gitlab.com/orderlynetwork/orderly-v2/strategy-vault/-/commit/2aa49089a49488d0b244aac72178b556b0e89c00>

LOW **L-02**

ACKNOWLEDGED

# Previous Rebalance Request Is Not Cleared

|          |                                                                                                                                                                                                                                                                                                                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Unexpected Behavior                                                                                                                                                                                                                                                                                                                                                               |
| LOCATION | <a href="https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/fdb4fd2e7284d882a74e93d6248387e0761fa545/contracts/Ledger/ProtocolVaultLedger.sol#L401C1-L404C46">https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/fdb4fd2e7284d882a74e93d6248387e0761fa545/contracts/Ledger/ProtocolVaultLedger.sol#L401C1-L404C46</a> |
| REVIEW   | Remediation Review                                                                                                                                                                                                                                                                                                                                                                |

## DESCRIPTION

When the owner requests a rebalance, the function is expected to clear the previous request and store the new one.

```
bytes32 rebalanceId = keccak256(abi.encode(vaultId, targetRatios, block.timestamp));  
// Clear and push rebalance request  
delete rebalanceRequest[rebalanceId];
```

However, the `rebalanceId` of a request is computed as `keccak256(abi.encode(vaultId, targetRatios, block.timestamp))`, meaning each request will have a different ID unless all parameters are exactly the same. As a result, the same vault and strategy providers can have multiple valid rebalance requests concurrently, since the previous request is not actually cleared.

Additionally, the rebalance request creation timestamp is neither emitted in an event nor stored in `storedRatios`, and the `rebalance` function does not take it into account. As a result, an earlier request can be executed after a later one, effectively overriding the most recent rebalance request.

For example:

- The owner creates a rebalance request with a 30:70 ratio for 2 SPs in the vault.
- Later, the owner creates a new rebalance request with a 40:60 ratio for the same 2 SPs, assuming it would clear the previous request.
- The previous request has not been cleared, so both requests remain valid.
- The operator executes the second request first, then executes the first request.
- As a result, the funds are rebalanced based on the 30:70 ratio.

## RECOMMENDATION

Consider having `requestId`s based only on `vaultId` and SP IDs, without including the ratios or `block.timestamp`. This way, there will be only one active request for the same vault-SP pair, even if the intended ratios change.

---

**L-02** Previous Rebalance Request Is Not Cleared

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

# Disabling strategy providers may DOS the Vault

CATEGORY Configuration  
LOCATION LedgerCoreImpl.sol#L154-162  
REVIEW Remediation Review

## DESCRIPTION

To disable a `strategyProvider`, first `isAllowedStrategyProvider` must be disabled in the `ProtocolVault` contract, then all in-flight messages must be executed and finally disable the `SP` in the `ProtocolVaultLedger` contract.

Otherwise the in-flight messages will be executed and the `OperationHandled` event will be emitted with a disabled `SP` which will lead to the reverting of `updateLPAndStrategyFund()` and DOS the strategy vault, since there the `SP` is checked against the allowed ones.

## RECOMMENDATION

Not recorded.

## RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

# Rebalance can assign assets to a 0-ratio SP

|          |                          |
|----------|--------------------------|
| CATEGORY | Unexpected Behavior      |
| LOCATION | LedgerExtension.sol#L170 |
| REVIEW   | Remediation Review       |

## DESCRIPTION

The `rebalance()` function treats the last `SP` as a fund that receives all of the assets leftovers after the previous `SPs` have been allocated to.

```
for (uint256 i = 0; i < len; i++) {  
  if (i < len - 1) {  
    targetAssets[i] = totalMainAssets.mulDiv(targetRatios[i].ratio, REBALANCE_BASE,  
Math.Rounding.Floor);  
    allocatedAssets += targetAssets[i];  
  } else {  
    targetAssets[i] = totalMainAssets - allocatedAssets;  
  }  
}
```

Since the `targetAssets` are rounded down for each fund, the leftover assets will be assigned to the last `SP` even if it's `targetRatio = 0`

## RECOMMENDATION

To avoid assigning funds to an `SP` with `targetRatio = 0`, make sure the last `SP` in the `rebalanceRequest` array has a positive `targetRatio`. This can also be enforced in the `[ProtocolVaultLedger.requestRebalance()]`(<https://github.com/GuardianOrg/orderly-v2-strategy-vault-orderlyvault-team1/blob/fdb4fd2e7284d882a74e93d6248387e0761fa545/contracts/Ledger/ProtocolVaultLedger.sol#L389-L411>) function.

## RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

# Rebalanced SP can end up having shares, but no assets

CATEGORY Unexpected Behavior  
LOCATION LedgerExtension.sol#194-198  
REVIEW Remediation Review

## DESCRIPTION

When `LedgerExtension` rebalances the assets in the `SP` funds, it modifies the `totalAssets`, `mainShares` and `totalShares`.

```
deltaAssets = currentMainAssets - targetAssets[i];  
deltaShares = deltaAssets * 10 ** USDC_DECIMAL / sharePrice;  
fund.totalAssets -= deltaAssets;  
fund.totalShares -= deltaShares;  
fund.mainShares -= deltaShares;
```

Since `deltaShares` are rounded down, `mainShares` can end up being slightly larger than expected. This can become problematic if there are no `SP` shares because it will result in the fund not having any assets - because they were fully subtracted, but still having dust shares. This will also impact the `hwm` because it won't enter the zero case during settlement

```
if (totalShares == 0) {  
    return 10 ** decimal;  
}
```

## RECOMMENDATION

If `targetAssets[i] == 0`, execute the following logic instead

```
fund.totalAssets -= currentMainAssets;  
fund.totalShares -= fund.mainShares;  
fund.mainShares = 0;
```

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

# quoteClaim Uses Param ChainId While Send Path Uses message.dstChainId

|          |                            |
|----------|----------------------------|
| CATEGORY | Validation                 |
| LOCATION | VaultCrossChainManager.sol |
| REVIEW   | Remediation Review         |

## DESCRIPTION

In the `VaultCrossChainManager` contract, `quoteClaim(uint256 chainId, StrategyVaultCCMessage message)` derives the destination endpoint ID (EID) from the function parameter `chainId`, whereas the actual send path `sendMessage` derives the destination from `message.dstChainId`. If a caller supplies a `chainId` that differs from `message.dstChainId`, the quoted LayerZero fee will be computed for one destination but the send will target another, producing incorrect fees.

Illustrative code:

```
// quoting path
function quoteClaim(uint256 chainId, StrategyVaultCCMessage memory message) public view
returns (MessagingFee memory) {
    bytes memory lzMessage = abi.encode(message);
    uint32 dstEid = chainIdToEid[chainId]; // uses param
    MessagingFee memory fee = _quote(dstEid, lzMessage, _getOptions(message.payloadType), false);
    return fee;
}

// send path
function sendMessage(StrategyVaultCCMessage memory message) external onlyLedger {
    bytes memory lzMessage = abi.encode(message);
    uint32 dstEid = chainIdToEid[message.dstChainId]; // uses message field
    MessagingFee memory fee = _quote(dstEid, lzMessage, _getOptions(message.payloadType), false);
    _lzSend(dstEid, lzMessage, _getOptions(message.payloadType), fee, payable(address(this)));
}
```

When the two values diverge, operators may underfund the send (leading to `NotEnoughNative` and failed delivery) or overfund. While current usage appears to keep them equal, the

**I-01** quoteClaim Uses Param ChainId While Send Path Uses message.dstChainId

CONTINUED

API remains a footgun that can surface as intermittent delivery failures or unnecessary balance management overhead.

**— RECOMMENDATION**

Consider removing the `chainId` parameter from `quoteClaim` and derive the destination solely from `message.dstChainId`.

**— RESOLUTION**

Status: Acknowledged.

## INFORMATIONAL I-02

ACKNOWLEDGED

# Unenforced Operation Order Enables Mis-sequenced Period Actions

|          |                         |
|----------|-------------------------|
| CATEGORY | Configuration           |
| LOCATION | ProtocolVaultLedger.sol |
| REVIEW   | Remediation Review      |

## DESCRIPTION

`ProtocolVaultLedger` exposes several operator entry points that must be run in a specific per-period order for accounting and custody to stay consistent. The contract only enforces "once per period" flags (e.g., `isAllocatedToFunds`, `isAssetDistributed`) and period equality (`_check(periodId)`) on some functions, but it does not enforce the required sequencing between them. Several calls accept any `periodId` and can be executed out of order without reverting.

Examples that compile and run but produce inconsistent state:

- allocate before NAV: Calling `allocateToFunds` before `updateStrategyFundAssets` uses stale `fundAssetsAfterFee/lastSharePrice` for per-fund conversions, mispricing shares.
- distribute without period gating: `distributeAssets` has no `_check(periodId)` and can be called for any period (including "after" `updatePeriodId`), decoupling custody movement from the accounting window.
- ...

Because of this, operators can unintentionally execute valid transactions in the wrong order, causing mispricing, inconsistent books vs custody and operational foot-guns that are hard to detect and fix.

## RECOMMENDATION

Enforce an explicit per-period state machine and add ordering preconditions:

- Add a stage enum per period (e.g., `None` → `NavUploaded` → `OpsProcessed` → `Allocated` → `Distributed` → `FundsSettled` → `AccountsSettled` → `Closed`).
- Gate each function with required stage(s).

## RESOLUTION

Status: Acknowledged.

## § 05

# Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

# About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

**\$45B+**DIGITAL ASSETS UNDER  
REVIEW**200+**

ENGAGEMENTS COMPLETED

**500+**CRITICAL & HIGH FINDINGS  
REPORTED

Reach us at [audits@guardianaudits.com](mailto:audits@guardianaudits.com).